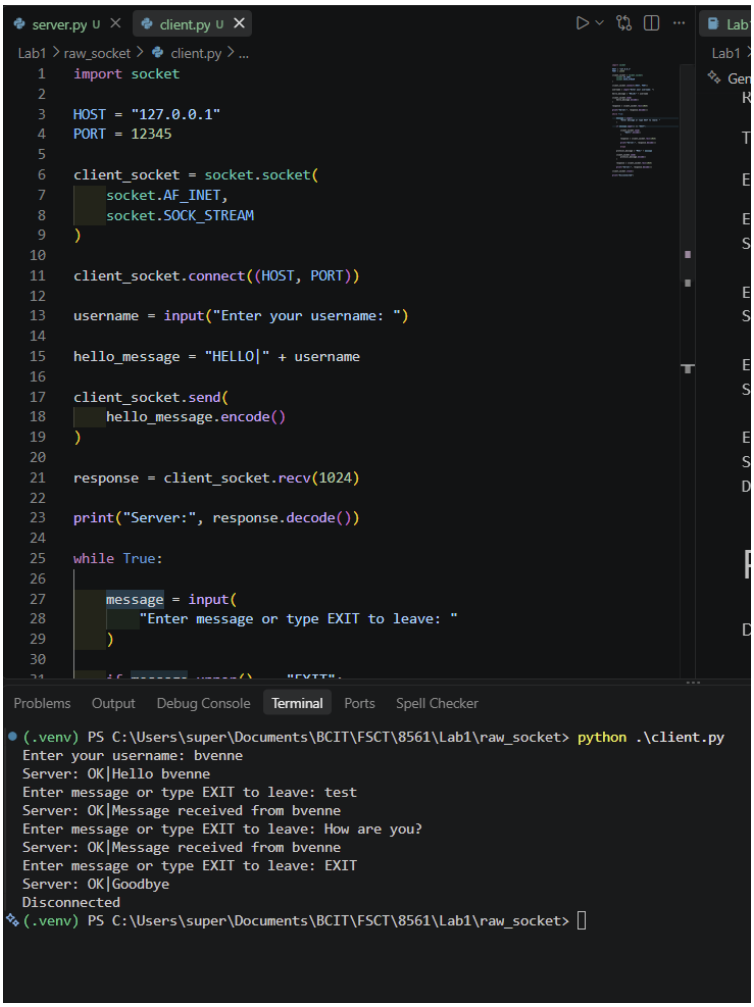
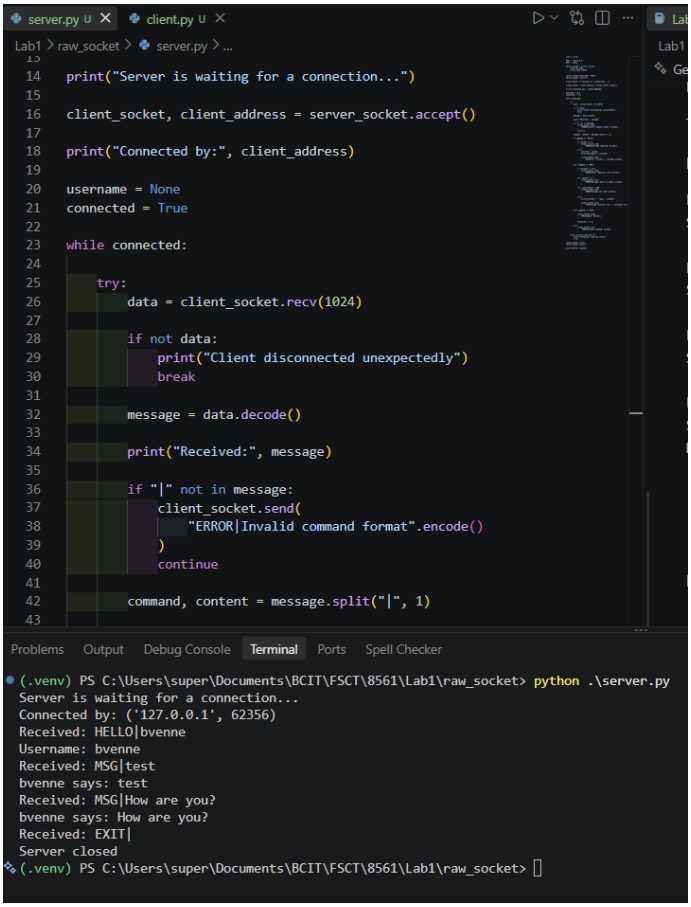
Braeden Venne

Lab 1

Part 7 – Test Normal Operation

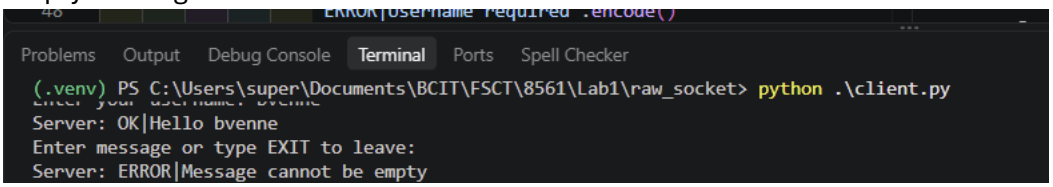


Braeden Venne

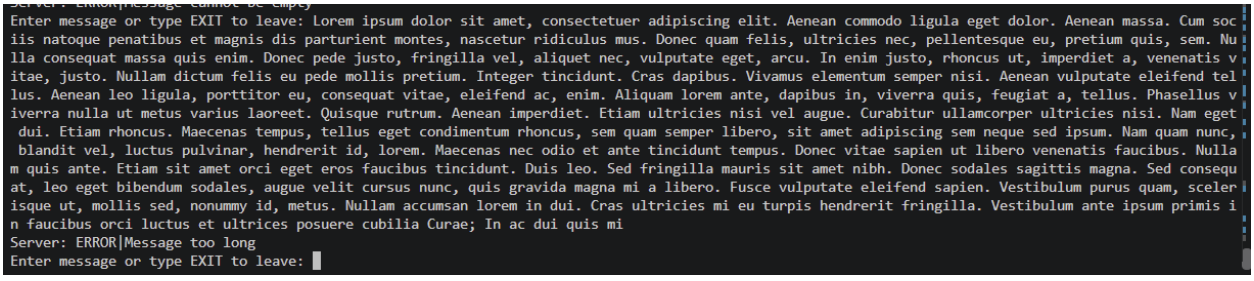


Part 8 – Robustness test

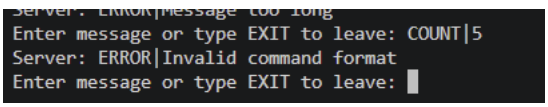
- Empty message



- Message longer than 200 characters



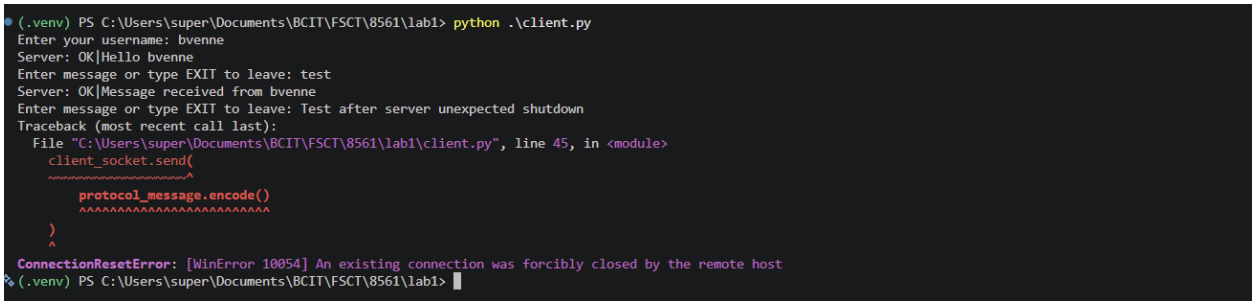
- Unknown command



Braeden Venne

Part 10 – Security Reflection

1. What happens if the server crashes while the client is connected?
    a. The Client's next message or send() / recv() will throw a ConnectionResetError stating that the existing connection was forcibly closed by the server.



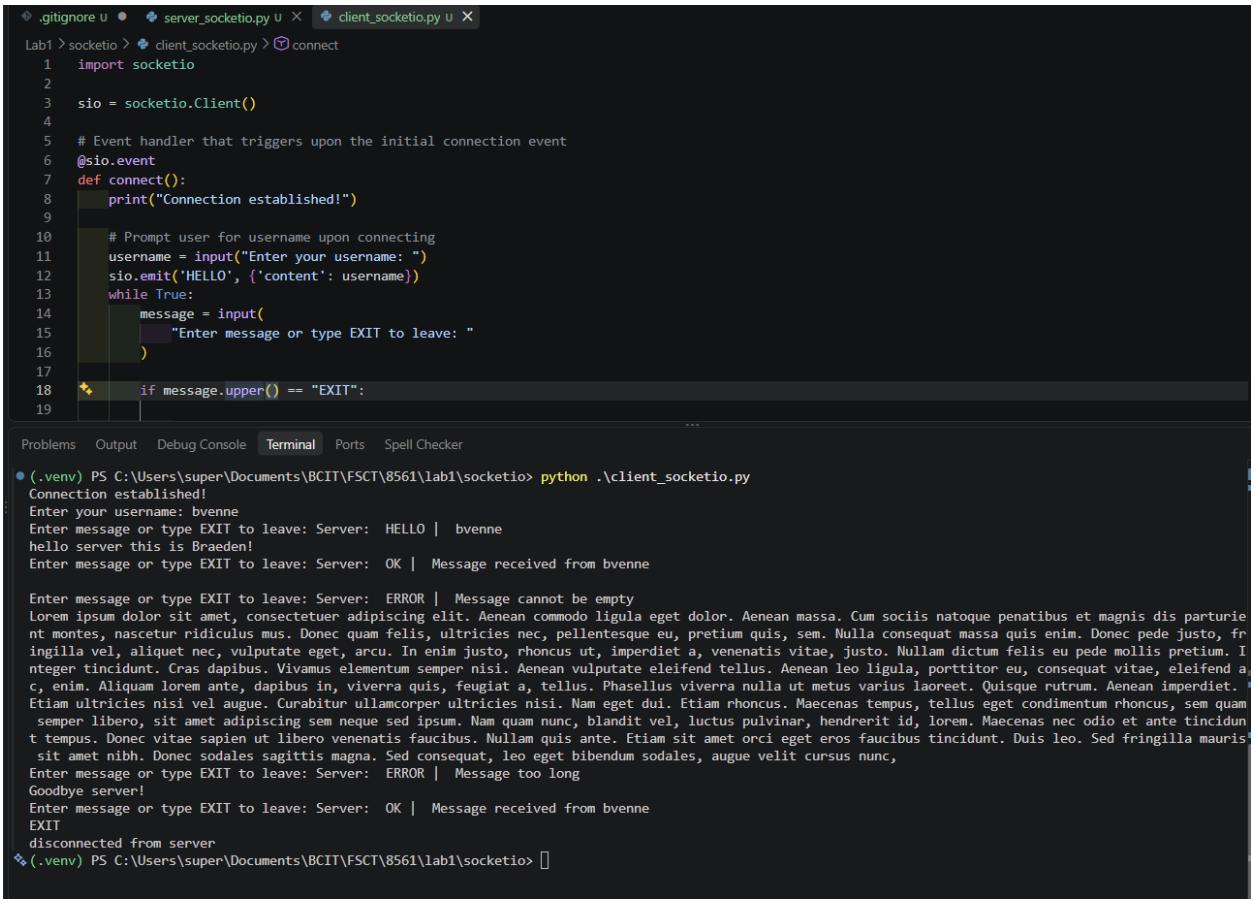
2. How could the server be extended to handle multiple clients?
    a. You would need to implement either a multi-threading python library, or you can setup Socket.io which handles it natively with an asynchronous runner.
3. Why can `recv(1024)` split messages unexpectedly?
    a. Messages may split unexpectedly because the 1024 value is only a maximum buffer cap, the recv function will still pull whatever partial data is sitting in the network buffer at the exact moment rather than waiting for full messages to arrive. Additionally, if a message is larger than 1024 bytes it will split due to the buffer cap.
4. Why does TCP reliability not mean the application is secure?
    a. While TCP does ensure that 100% of your data reaches the destination, it does not handle things like encryption and authentication for you, making it susceptible to Man-in-the-Middle attacks or impersonation.
5. What new attack surfaces are created by adding usernames and protocol commands?
    a. There is a risk of command injections or application crashes if unexpected characters or super long strings are sent as the username
6. How could a malicious client abuse the protocol?
    a. Theye could do impersonation/spoofing as usernames are self-declared without any sort of validation/authentication. Malicious clients could also flood the server with messages greatly exceeding the 200-character limit to exhaust the server buffers.
7. How would you add authentication?
    a. I would create a secure database and have the users require a username/password pair. The server would then check this stored username/password pair and utilize this for authenticating the user's session.
8. How would you add encryption?
    a. For implementing encryption, I would combine the ssl python library with the socket library which provides a socket-like wrapper that also encrypts and decrypts the data going over the socket with SSL (https://docs.python.org/3/library/ssl.html). Alternatively, if using Socket.io I can change my connection string from http to https.
9. What validation is performed in your implementation?
    a. This implementation validates the following

i. command structure
ii. ensures a username is entered before a message is accepted
iii. message/username emptiness
iv. chat messages are limited to a maximum of 200 characters

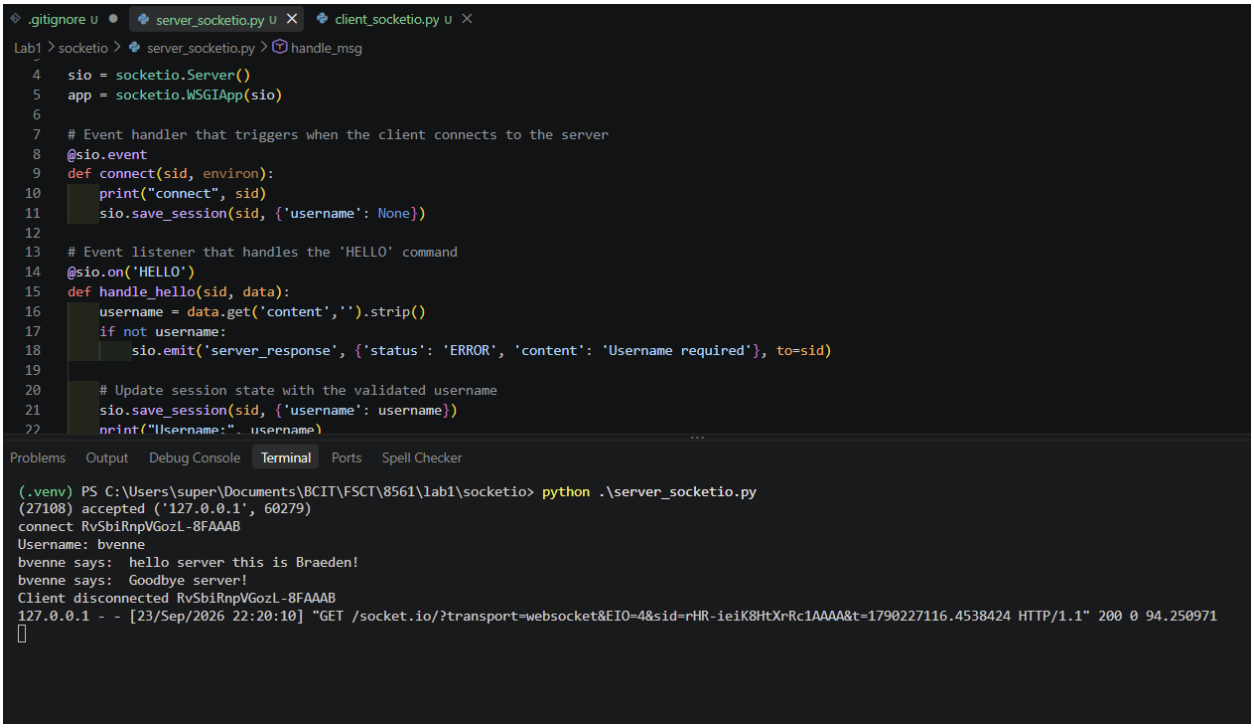
10. What additional mitigations would be needed for a real deployment?
    a. You would need mitigations such as more robust error handling, rate limiting, and session expiration.

Part 11 – Raw TCP vs. Socket.io

Used https://python-socketio.readthedocs.io/en/stable/server.html for a lot of code reference



Braeden Venne



Discussion:

Socket.IO handles connections, sending/receiving messages, and event handling, eliminating the need to use send(), recv(), or bind(). Overall, Socket.IO offers a simpler/easier code to write whereas our TCP implementation is more complex but is more beneficial for understanding exactly how the client-server communication works.

Braeden Venne

Challenge – Support Multiple Clients

Initial Connection



Braeden Venne



```
client.py U  server.py ...\challenge U  socketio-server.py U  socketio
Lab1 > challenge > client.py > ...
1   import socket
2   import threading
3
4   HOST = "127.0.0.1"
5   PORT = 12345
6
7   # Create the client socket
8   client_socket = socket.socket(
9       socket.AF_INET,
10      socket.SOCK_STREAM
11  )
12
13  # Connects to the server
14  client_socket.connect((HOST, PORT))
15
16  # Prompt user for their username
17  username = input("Enter your username: ")
18
19  # Create 'HELLO' message
20  hello_message = "HELLO|" + username
21
22  # Send username to the server
23  client_socket.send(
24      hello_message.encode()
25  )
26
27  # Receive the welcome response from the server
28  response = client_socket.recv(1024)
29
30  print("Server:", response.decode())
31
32  # Handles messages received from the server
33  def handle_messages():
34      while True:
35          data = client_socket.recv(1024)
36
37          if not data:
38              break

Problems 1  Output  Debug Console  Terminal  Ports  Spell Checker 1

(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab1\challenge> python .\client.py
Enter your username: Alice
Server: OK|Hello Alice
Enter message or type EXIT to leave:
```



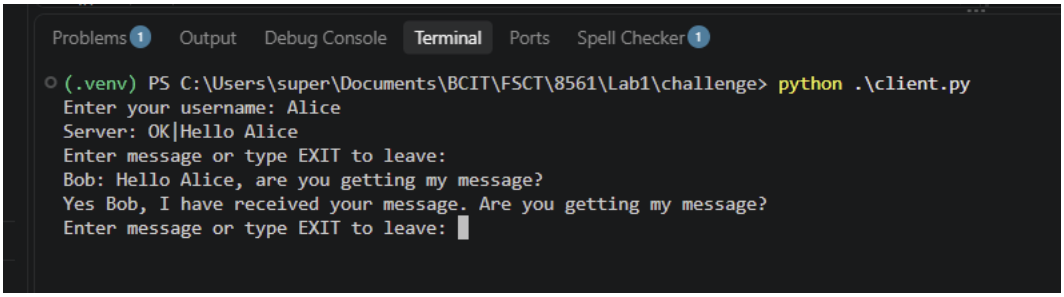
```
client.py U  server.py ...\challenge U  socketio-server.py U  socketio
Lab1 > challenge > client.py > ...
1   import socket
2   import threading
3
4   HOST = "127.0.0.1"
5   PORT = 12345
6
7   # Create the client socket
8   client_socket = socket.socket(
9       socket.AF_INET,
10      socket.SOCK_STREAM
11  )
12
13  # Connects to the server
14  client_socket.connect((HOST, PORT))
15
16  # Prompt user for their username
17  username = input("Enter your username: ")
18
19  # Create 'HELLO' message
20  hello_message = "HELLO|" + username
21
22  # Send username to the server
23  client_socket.send(
24      hello_message.encode()
25  )
26
27  # Receive the welcome response from the server
28  response = client_socket.recv(1024)
29
30  print("Server:", response.decode())
31
32  # Handles messages received from the server
33  def handle_messages():
34      while True:
35          data = client_socket.recv(1024)
36
37          if not data:
38              break

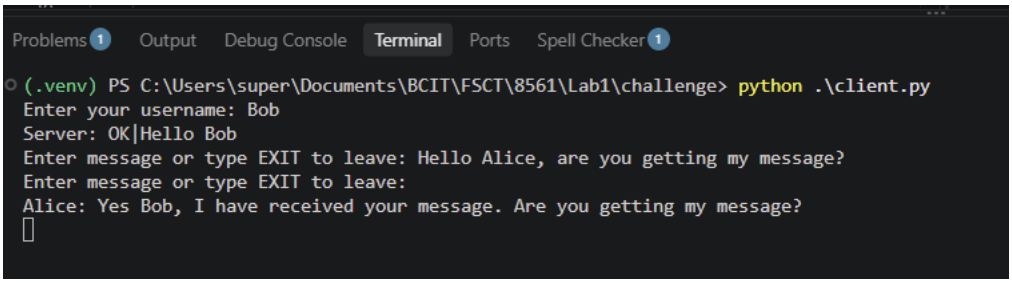
Problems 1  Output  Debug Console  Terminal  Ports  Spell Checker 1

(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab1\challenge> python .\client.py
Enter your username: Bob
Server: OK|Hello Bob
Enter message or type EXIT to leave:
```

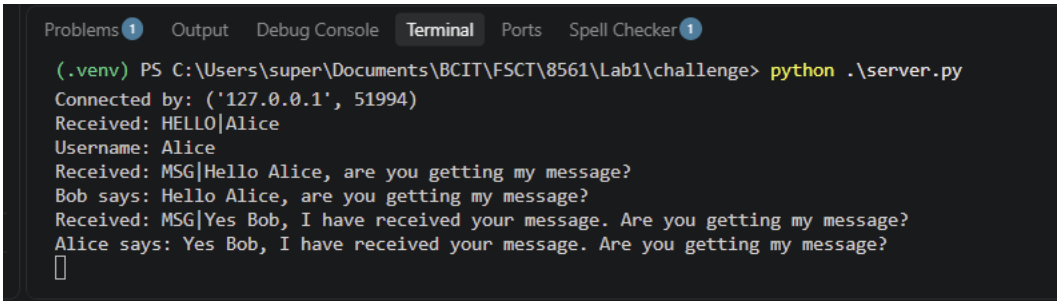
Braeden Venne

Sending messages between Alice and Bob – Client POV





Sending messages between Alice and Bob – Server POV



Challenge Questions:

1. Why could the original Lab 1 server handle only one client at a time?
    a. The original server did not utilize threading, and would get stuck inside the message loop, preventing it from accepting any new client connections.
2. What problem does threading solve?
    a. Threading allows for the server to handle multiple connected clients independently on a separate background thread. This keeps the main loop open allowing additional clients to connect simultaneously.
3. What happens to the other clients when one client disconnects?
    a. The other clients are unaffected and are still able to keep sending messages as intended.
4. What additional security risks arise when multiple clients share the same server?
    a. A malicious client or bad actor could spam messages to flood the server buffer, triggering a DoS for everyone else.
5. What could happen if two clients try to use the same username?

    a. Since this implementation does not have a validation check for unique usernames, it will allow users to have the same names. All this will mainly cause is confusion as it will look like the same person is sending messages to themselves.
6. How would you prevent one user from impersonating another user?
    a. I would add another validation check on the server that stores active usernames, the server can then check/verify if the username already exists and return an error to prompt the user to try another name as that one is in use.

Short Security Analysis

In this lab we built a chat server which highlighted major security gaps in raw socket applications in comparison to frameworks such as Socket.IO. While our raw TCP setup works under normal operations, it lacks serious network protections, making it vulnerable to resource attacks and spoofing/impersonation.

A major issue in the TCP implementation is how it handles basic buffer streams. Due to raw TCP being stream-oriented, reading data with a hard capped recv(1024) can cause messages to split unexpectedly if the maximum buffer size is exceeded. This creates an attack surface where a malicious client can flood the socket with massive messages, exhausting the server's resources and memory buffers. Additionally, the server accepts any self-declared username without any means of verification or authentication. This makes it severely vulnerable to impersonation and spoofing. Multiple users can claim identical usernames, generating data confusion. While we did implement threading in this lab to mitigate the issue where only a single client can access the server at a time, this introduces additional security risks. There would need to some sort of authentication set in place, where the server stores a unique username and password pair and validates the user's session.

Ultimately, TCP reliability does not mean application security. While it does 100% guarantee the packets get delivered to their destination, it does nothing to offer protections against Man-in-the-Middle attacks or data tampering. To fix this I would implement encryption by combining the ssl python library with the socket library. This provides a socket-like wrapper that encrypts and decrypts the plaintext data going over the socket with SSL.